The Performance Commons

Why ASIC-level performance is a collective-action problem, and the hardware we should build for its solution

A position paper. The underlying architecture analysis is drawn from the companion technical comparison, "Dataflow vs. an Optimal Heterogeneous Architecture."

Abstract

The reason we don't achieve near ASIC performance and power for many more applications (and the reason we don't have much better ASICs) is not because it's technically impossible given the engineering resources available. It is the collective-action problem both within and across levels of the computing stack. As machine intelligence continues to advance — as long as it maintains the apparent inoculation to this problem — this fact will increasingly be exposed. This raises many questions for forward looking engineers in the form of, "How do we design systems assuming the collective-action problem is solved?". For example, how do we design hardware assuming there will be orders of magnitude more intelligence available for shared code optimization? We likely want more reconfigurable and less "programmable" architectures. And similarly, how do we design software stacks assuming the programmability issues holding back wider adoption of distributed computing are solved?

1. The gap is real, and it is not physics

Start by splitting a concept the field treats as one thing. An architecture's "programmability" is two different numbers: the performance it yields at *constant, ordinary programmer effort* — what a competent team gets by default — and the best performance *achievable* on it, its **realizable frontier**, the ceiling the roofline model draws per kernel (Williams, Waterman & Patterson, CACM 2009). Comparing architectures to ASICs measures neither; the number that matters first is the distance between the two on a *fixed* architecture — the **realization gap** — and it is measured, on unchanged silicon. A 4096×4096 matrix multiply runs **more than 60,000× faster** hand-optimized than in naive Python, from 0.0006% to 41% of machine peak (Leiserson et al., Science 2020), and Intel measured a **"ninja gap" averaging 24×** between naively written and expert-optimized C++ on the same processor (Satish et al., ISCA 2012). Most general-purpose code runs one to two orders of magnitude below its own architecture's frontier — full HPC applications, already tuned harder than most software, measure at 4–14% of peak on commodity platforms (Oliker et al., SC 2004).

Nor is the frontier the industry does reach optimal. The M1 matched the best x86 single-thread performance at roughly a fifth the power — the same general-purpose programmability, strictly better, no tradeoff paid (AnandTech, 2020); Etched claims the same inference function at under half the industry's voltage for multiples of the FLOPs density (vendor claims; §5 tells both stories in full). A design point that can be dominated without surrendering programmability was never on the true frontier at all.

What physics actually charges for generality is a separate and smaller number, also measured: a general-purpose core is roughly **100–500× less energy-efficient than an ASIC** for the same task (Hameed et al., ISCA 2010); word-level reconfigurable hardware (a CGRA) sits within ~2.8× of an ASIC; bit-level reconfigurable hardware (an FPGA) ~20–35× off (Prabhakar et al., ISCA 2017; Kuon & Rose, 2007); a GPU lands roughly 3–15× off a node-normalized ASIC in energy on the regular data-parallel kernels it is built for (Chung et al., MICRO 2010), and Google's first TPU delivered ~30× the performance-per-watt of its contemporary, pre-tensor-core GPU on datacenter inference (Jouppi et al., ISCA 2017). That GPU tax is collapsing where instructions amortize: with tensor-core matrix instructions, 77–87% of the energy goes to the arithmetic itself, bounding a dedicated accelerator's remaining advantage on the core matrix multiply at 13–23% — near-ASIC efficiency for the work the GPU specializes (Dally, Turakhia & Han, CACM 2020). Etched's under-half-voltage claims mark that accounting's limit, though: instruction overhead is only one tax, and a fixed operating point leaves voltage-level co-design on the table (§5). The companion analysis shows a perfect compiler and scheduler brings each substrate to its roofline, and the classic limit studies (Lam & Wilson, ISCA 1992; Wall, ASPLOS 1991) locate the dominant barrier in *control flow and the effort of exposing parallelism* — not in raw data dependence. What separates achieved from achievable is therefore, overwhelmingly, the **effort of specialization** — writing the optimal kernel, building the accelerator, tuning to the platform — and not a wall in the silicon.

Substrate	Penalty vs an ASIC (same function)	Measured by
ASIC	1× — the efficiency ceiling	—
CGRA (word-level reconfigurable)	~2.8× area, on parallel-pattern kernels	Prabhakar et al. (Plasticine), ISCA 2017
FPGA (bit-level reconfigurable)	~20–35× area, ~3–4× delay, ~10–14× power	Kuon & Rose, 2007
GPU (SIMT, software-programmable)	~3–15× energy on regular data-parallel kernels; ~30× perf/W behind a same-era inference ASIC	Chung et al., MICRO 2010; Jouppi et al. (TPU), ISCA 2017
General-purpose CPU	~100–500× energy	Hameed et al., ISCA 2010

Table 1. The measured cost of generality. These are the intrinsic, physical penalties — the realizable frontier of Figure 1 — not what most code actually achieves, which is typically worse because the substrate's potential goes unextracted.

2. The general-purpose vs. custom trade-off is mostly an effort artifact

The trade-off at the center of computer architecture is usually drawn as a Pareto frontier between **flexibility** and **efficiency**: a CPU does anything but wastes energy; an ASIC wastes nothing but does one thing; FPGAs and CGRAs sit in between. The conventional reading is that you *buy* efficiency with flexibility and pick your point on the curve. But that curve is drawn at a fixed, finite *programmer effort* — it is a slice through a three-dimensional surface (flexibility × efficiency × effort), and the slice moves as effort changes.

To see why, decompose the efficiency gap between a reconfigurable substrate and an ASIC into two parts. The first is the **intrinsic fabric tax**: the physical overhead of the reconfigurable silicon itself — the switches, the configuration memory, the over-general datapath. This is irreducible; no amount of programmer effort removes it, and it is what the measured ratios of Table 1 capture. The second is §1's **realization gap**: how much of the substrate's *available* efficiency you actually extract, which depends entirely on the quality of mapping, placement, and scheduling — and therefore on effort. At finite effort you extract a fraction; at the limit (the oracle mapping of the companion report) you extract all of it.

The consequence is the whole argument in one picture. As effort rises, the realization gap closes and the achieved-efficiency curve falls toward the realizable frontier — which is just the intrinsic fabric tax. Crucially, **the effort gap is concentrated exactly on the reconfigurable substrates**: it is ~zero at the custom end (an ASIC has nothing to program), and §1 measured it at one to two orders of magnitude even at the general-purpose end, where compilers are most mature — so it can only be larger in the middle, where the tooling is youngest and the mapping hardest. That concentration is inferred rather than measured — no published study pins the achieved-versus-achievable gap on an FPGA or CGRA, which is part of what Appendix A's experiment is for, and the open CGRA toolchains that do exist are fragmented, routinely failing to map even simple kernels, with no LLVM-equivalent shared infrastructure formed (Walter et al., 2025) — §4's unpooled state, measured in tooling. But the signature is public: reconfigurable fabrics have historically underdelivered not because they are inefficient, but because their efficiency was unreachable at affordable effort. Close that gap — which is what a shared, AI-driven optimization commons does — and a CGRA sits within ~2.8× of an ASIC *while keeping full flexibility* — a tax measured on the regular, parallel-pattern kernels Plasticine was evaluated on; how far it extends across the workload space, especially control-flow-heavy code, is exactly the boundary Figure 6's phase diagram names and Appendix A's simulator sweep is built to measure.

The scoping already has one strong answer on record: a composed, programmable architecture — tiny cores with configurable spatial fabric, scratchpads, and DMA — matched the speedups of four diverse domain-specific accelerators (10–150× over an out-of-order core) at no more than ~4× the area and power of a single one, under a title that states the thesis outright: *domain specialization is generally unnecessary for accelerators* (Nowatzki, Gangadhar, Sankaralingam & Wright, HPCA 2016; IEEE Micro Top Picks 2017). The line has since been mechanized — accelerators decompose into a few spatial primitives that tools compose automatically (DSAGEN, ISCA 2020), and generated *programmable* overlays already match untuned fixed-function HLS, reaching 0.55× of hand-tuned (the honest residual; OverGen, MICRO 2022) — and independently replicated: for analytics, one programmable systolic design matched a bundle of specialized accelerators (Lottarini et al., ISCA 2019). Sankaralingam co-authored the dark-silicon result that launched the specialization era (Esmaeilzadeh et al., ISCA 2011); the line that started the race concluded the finish was general. The flexibility–efficiency trade-off does not so much shift as **flatten**: under unlimited effort, the only efficiency you still surrender for flexibility is the small, physical fabric tax.

One more turn of the screw closes the loop with the design conclusion. Even *unlimited design effort* — machine intelligence spinning up bespoke ASICs cheaply — does not argue for ASICs everywhere, because an ASIC is fixed at tape-out and real workloads are not: the adaptability an ASIC structurally lacks is exactly what its efficiency edge over a CGRA costs. So unlimited effort, applied honestly, does not push the optimum toward *more custom* silicon; it pushes it toward **reconfigurable** silicon that an oracle-grade commons can drive to within a small factor of custom while remaining able to change what it computes. That is the analytical core of "more reconfigurable, less programmable." The physics record backs the direction of the screw: across thousands of chips, most measured accelerator gains came from CMOS scaling rather than architectural specialization — Bitcoin ASICs improved 307× from process and 1.7× from specialization — so with scaling slowed, specialization's returns run into an "accelerator wall" (Fuchs & Wentzlaff, HPCA 2019).

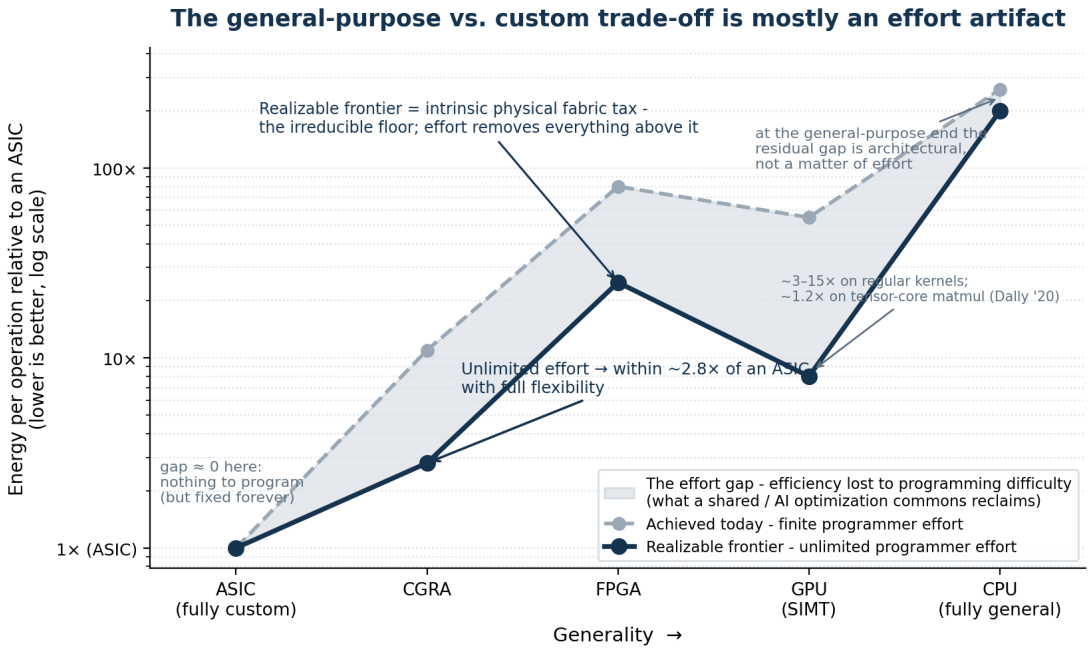

Figure 1. The canonical flexibility–efficiency trade-off is largely an artifact of finite programmer effort. The shaded band is the effort gap — efficiency forfeited because extracting a reconfigurable substrate's potential is too laborious to afford per workload. It is widest exactly where reconfigurable hardware sits and ~zero at the custom end (nothing to program); at the general-purpose end, the distance that remains at full effort is architectural — the fabric tax — not forgone effort. A shared / AI optimization commons reclaims the band; what remains is the intrinsic physical fabric tax (CGRA ~2.8×, FPGA ~20–35×). Substrate-to-ASIC ratios on the realizable frontier are anchored to measured values; the finite-effort curve and the effort axis are illustrative.

3. Near-oracular performance is a privilege

Consider who actually reaches the frontier today: hyperscalers with custom silicon, high-frequency-trading firms, AAA game-engine teams, codec and DSP vendors, and the handful of groups who hand-write CUDA or assembly. They get there through enormous, bespoke, and largely unshared effort, justified only by scale or stakes. Everyone else ships code far off the frontier — not for lack of talent or physics, but because the cost of specialization is affordable only at those scales. Performance is gated by *who can pay the specialization cost*, not by what is achievable.

4. The privilege is a collective-action problem

An optimization is a **non-rival good**: my hand-tuned FFT does not diminish yours. Classic economics says non-rival goods are under-provided whenever no single actor captures enough of the aggregate benefit to justify producing them for the whole population (Samuelson, 1954; Olson, 1965). So each team re-derives or hoards, and the population sits below the frontier. This is the textbook structure of a collective-action problem — under-provision of a public good. The performance gap is not a technology gap; it is a coordination failure.

Look at what the equilibrium actually selects: good enough. Most software ships at the first performance that stops hurting and stays there — satisficing, in Simon's original sense (Simon, 1956). Notice what kind of failure that is. For widely shared kernels the optimum usually *exists* — some team somewhere has paid for it (§3) — so there the failure is distribution, not invention; for the long tail nobody has paid, and what binds is a production cost §§8–9 show collapsing. Either way, in a free market with perfect information the gap could not persist: an implementation that is 10× cheaper to run is pure arbitrage, and arbitrage closes. It persists because the market for optimizations is missing, and it is missing for reasons economics has long named: the good is non-rival and hard to price; its quality is unverifiable to the buyer, which unravels the market that would trade it (Akerlof, 1970); and the transaction costs of selling an improvement per instance exceed the per-instance value (Coase, 1937; Arrow, 1969). Creating the market that would fix this — the verification machinery, the matching, the trust — is itself a fixed-cost public good that no single actor is positioned to provide: the collective-action problem recurses into the creation of markets themselves. Machine intelligence attacks every input of market formation at once — verification above all, since making quality legible is what makes it priceable (§11's inference-proposes-verification-disposes machinery), alongside matching and transaction cost — so the prediction is not only that the existing commons deepens, but that missing markets form.

The pieces of this diagnosis are each published; the synthesis is not. Leiserson et al. (Science 2020) locate post-Moore performance at the top of the stack — and concede that cross-layer gains come easiest inside "big system components" where one organization pools the costs and benefits; Thompson & Spanuth (CACM 2021) supply the economics of computing's fragmentation into specialization; Hooker (CACM 2021) the path-dependence of research on incumbent hardware; Hess & Ostrom (2007) the theory of knowledge commons; and Fursin's collective-optimization program (Fursin & Temam, TACO 2010) even built an optimization commons — before there was machine intelligence to fill it. What none of them state is the joint claim made here: the performance gap is an under-provided public good, and machine intelligence changes the provisioning economics.

5. The herd is what the failure looks like today

Watch the failure operate today and it does not look like teams re-deriving the same optimization in parallel; it looks like teams *copying* each other. Leadership everywhere sets the same safe target, because deviating carries individual career risk while conformity's failures are shared — herding is individually rational under reputational incentives (Scharfstein & Stein, AER 1990) — and inside the firm the same arithmetic actively suppresses the new: the champion of a novel direction bears its risk personally even when trying it would cost almost nothing, so near-free exploration goes unworked. The result is measurable monoculture. Convergence on one approach reduces collective welfare even when that approach is individually best (Kleinberg & Raghavan, PNAS 2021); research directions win by fitting incumbent hardware rather than by merit (Hooker's hardware lottery, CACM 2021); and independently built LLMs from different labs now produce convergent outputs and correlated errors — errors that grow *more* similar as capability rises (Jiang et al., NeurIPS 2025; Kim et al., ICML 2025). An industry's worth of private effort, and the products are all about the same.

The returns to defecting are exactly what this predicts. Apple's M1 matched or beat the best x86 single-thread performance at roughly a fifth the power (AnandTech, 2020) — against Zen 3 parts fabbed one node behind the M1's N5, a step worth tens of percent in power, not fivefold — and lifted Mac revenue ~40% in two years ($28.6B to $40.2B, FY2020–22 10-K filings) — a gain of that size is not an integration dividend but a ground-up rearchitecting of the machine (documented from the outside since Apple itself has said little), which vertical integration merely permitted Apple to attempt alone, exempt from the stack's collective-action problem. The herd then followed. DeepSeek's R1 erased $589B of NVIDIA's market value in a single day (January 2025) with engineering that its technical reports document line by line — FP8 training, latent attention, hand-written PTX communication kernels compensating for the export-cut interconnect of its H800s (DeepSeek-AI, Nature 2025). That is the specialization effort the unconstrained herd declines to pay, paid under duress — and it recovered most of the performance. And the pattern reaches the silicon itself. Voltage scaling is what every undergraduate computer-architecture student learns, with decades of near-threshold literature behind it (Dreslinski et al., Proc. IEEE 2010; Intel demonstrated a near-threshold x86 at ISSCC 2012) — yet it took a startup co-designing from transistor to token to run its math blocks at under half the voltage of incumbent AI chips, claiming multiples of their FLOPs density and 80%+ sustained utilization where GPUs deliver 20–50% (Etched, 2026 — vendor claims, first racks shipping). A textbook technique sat unexploited on billion- and trillion-dollar roadmaps until a defector picked it up.

Each of these defections required a fortune to attempt — a trillion-dollar company's integrated stack, a stockpiled GPU fleet, an $800M war chest — because defying an industry's encoding means re-deriving, alone, what the herd will not pool. Hence this paper's central prediction: the entry fee is temporary. The rest of this section names what actually binds the herd; §8 and §9 measure the collapsing cost of leaving it. It will not take a billion dollars to reproduce these results for long.

To be fair to the herd, incentive is only half the mechanism — the other half is that most of the people involved have little idea why anything works or is the way it is. The same Etched announcement shows what deviating actually costs: lowering the voltage meant redesigning much of the stack around it — math arrays, tiling and scheduling, power delivery, packaging, cooling — because in a mature design every choice is load-bearing for the others. Engineers inherit artifacts whose rationales left with their authors, and when no one understands the system end to end, each reuse of the last design is individually prudent — the only move that does not require re-deriving a web of interlocking decisions nobody fully holds. The timidity is still real, but it belongs to no individual: it is encoded in the social mechanisms and in the competencies available to them, and the institution is timid even when none of its members are. This is the collective-action problem at its deepest — not just optimizations going unpooled, but understanding itself. And that is exactly this paper's argument: the encoding is changing. Machine intelligence raises the ambient competence — holding the whole stack's *why* at once — far enough that designing the right thing becomes cheaper than reusing the last one.

The chip embargo is the natural experiment. It aimed to preserve "as large of a lead as possible" in compute (Sullivan, 2022), and the compute gap held — a ~4-year training-hardware lead by Epoch AI's estimate — while the capability gap collapsed anyway, from double digits in 2023 to low single digits by 2026 (Stanford AI Index): the open-weights Kimi K2 Thinking came within a point of the closed frontier on Artificial Analysis's independently run index, and its successor K3 sits in the Arena top ten, statistically tied with US frontier models and first on the blind frontend-coding evaluation. Distillation, stockpiled GPUs, and subsidy are real confounders, and the constrained labs herd too — ten o1-style clones shipped within four months of o1. But the narrow inference survives all of it (distillation cannot copy R1's line-by-line documented engineering, and the Kimi models are original training runs, not copies): the binding constraint on the herd was never the silicon. It was the effort and the variance the herd declines to spend — and the labs that were forced to spend it, and chose to publish the result as open weights, turned constraint into a commons contribution.

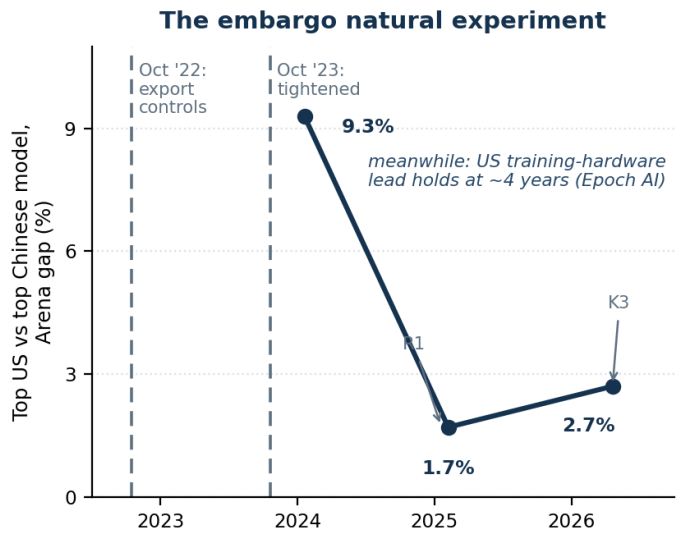

Figure 2. The embargo natural experiment, drawn. The gap between the top US and top Chinese model on the LMArena leaderboard collapsed from 9.3% (January 2024) to 1.7% (February 2025) and 2.7% (early 2026; Stanford AI Index 2025, 2026), across a period in which US export controls — October 2022, tightened October 2023 — held the training-hardware gap at roughly four years (Epoch AI). Chips were successfully restricted; capability parity happened anyway. Single-metric series; leaderboard evaluations have known limits.

Beneath the herding sits what the author's companion coherence work names the bottleneck: wanting itself. Much of the leadership involved does not, operatively, want the better product — and the evidence is that they say so. Of 401 surveyed financial executives, **78% admitted they would sacrifice long-term economic value to smooth reported earnings**, 80% would cut R&D, advertising, or maintenance to hit an earnings target, and a majority would delay a value-creating project to make the quarter's number (Graham, Harvey & Rajgopal, 2005). Behavior matches the stated intent: managers insulated from discipline close fewer old plants and open fewer new ones — the "quiet life" (Bertrand & Mullainathan, JPE 2003) — and buy personal safety with shareholder money, through diversifying acquisitions the market prices negatively on announcement (Gormley & Matsa, JFE 2016). Innovation is the systematic casualty, as agency theory predicts for high-variance work (Holmström, 1989): firms bunching at earnings forecasts cut R&D and innovate measurably less — a distortion estimated to raise firm value ~1% while costing ~1% of social welfare (Terry, Econometrica 2023) — and going public alone cuts innovation novelty by roughly 40% while the best inventors leave (Bernstein, J. Finance 2015). Privately rational, collectively wasteful: the under-provision above, written into executive behavior. The sharpest confirmation runs through the career-risk mechanism itself — where institutional owners absorb the personal risk of failure, innovation *rises* (Aghion, Van Reenen & Zingales, AER 2013). The binding variable is the individual risk the herd exists to manage.

There is a subtler casualty of the competence deficit. A leader who cannot evaluate the work itself still needs a proxy for productive value, and the proxy that survives incompetence is subjugation: compliance, visibility, deference to process — the signals any leader can read. This is measurement theory, not malice — when output cannot be measured, organizations control behavior instead (Ouchi, Management Science 1979; Holmström & Milgrom, 1991) — but the proxy selects against exactly what new work requires, because the people doing something genuinely new are, by construction, the least legible to the hierarchy above them. Doing new things requires freedom the proxy cannot grant. That piece has to go away too, and it goes away the same way the opacity does: when the work itself becomes legible, obedience loses its role as the measure of value.

They get away with it because the information does not flow. The counterfactual product, the delayed project, the value knowingly traded away are invisible to the boards and shareholders to whom the duty is owed; conduct that sits poorly beside fiduciary obligation survives on opacity, because quarterly numbers are legible and forgone alternatives are not. That asymmetry is ending on the same schedule as the rest of this paper's argument: machine intelligence reads what an organization actually did — its filings, its records, its code, increasingly its decision history — and surfaces what leadership optimized for, as the author's companion work on open organizations argues in full. Herding was always a wager that no one could see the counterfactual. The prediction is that the wager stops paying: the same intelligence that is inoculated against the coordination failure also makes it legible.

One market consequence is already visible at the top of the stack. Hardware incumbents hold their claim on the world's limited production capacity — the leading-edge wafers, packaging, and memory everyone else queues for — on a perception of technical expertise: scarce capacity flows to the firms believed uniquely able to convert it into value. Ambient competence dissolves exactly that belief, and the test becomes more stringent: not whether a firm can execute, which stops discriminating once everyone can, but whether it has the best story — Thiel's old criterion, that most of a company's value lies a decade out and what is priced is the account of why it will still be differentiated then (Thiel & Masters, Zero to One, 2014). Thiel has since made the mechanism explicit — AI looks "much worse for the math people than the word people" (Conversations with Tyler, 2024) — and the market is already pricing it: job postings for "storytellers" doubled in a year (LinkedIn, 2026). A story that survives ambient competence has to admit that technical capability is no longer the bottleneck and argue from what is still scarce: density of talent, and of genuinely wanted ideas — the argument Apple made in its best years. And the story can no longer merely be told; now that the work is legible, it has to survive reading.

Underneath all of it is the question of which thoughts are allowed to be thought. A herd's encoding does not merely price deviation; it defines what a serious person may work on, and preference falsification keeps the range narrow even in private (Kuran, *Private Truths, Public Lies*, 1995). What machine intelligence changes first is not the organization but the individual: one person with a competent collaborator no longer needs the hierarchy's permission — or its competence — to think a thought through to a working artifact. That is an increase in independence, and independence is the input this whole system has been starving for. Whether machine intelligence amplifies consensus instead is the fork §9 and §12 hold open.

6. The same logic explains the absence of better ASICs

One level down, ASIC *design and optimization* knowledge — mapping heuristics, verified IP blocks, the place-and-route and microarchitectural tricks that separate a good datapath from a mediocre one — is equally non-rival and equally hoarded. EDA vendors are a partial, proprietary aggregation point; the rest is re-derived bespoke per chip and locked inside firms. So the field produces fewer and worse ASICs than it collectively could, for the same reason: the design effort is not pooled. The size of the unpooled effort is public — leading-edge design NRE runs from ~$50M at 28 nm to ~$540M at 5 nm and $0.5–1.5B at 3 nm (IBS, via SemiEngineering, 2018), dominated by exactly the verification and software engineering that is re-derived per chip.

This is the aggressive claim, so state the caveat plainly: fab economics and the genuine bespokeness of different computations are also real costs, and pooling cannot erase them. The claim is *not* that the collective-action problem is the only barrier — it is that the **pooling-addressable fraction** of the gap is large and currently wasted.

Layer of the stack	The hoarded / secret non-rival good	Open-commons counter-trend
Foundry / process	PDKs & device models under NDA	SkyWater & IHP open 130 nm PDKs
CAD / EDA tooling	Synthesis, place-&-route, timing heuristics and their new RL optimizers (Synopsys/Cadence/Siemens; DSO.ai, Cerebrus)	OpenROAD, OpenLane, Yosys
Hardware IP blocks	Verified IP licensed proprietary	OpenCores, CHIPS Alliance, OpenHW CORE-V; Rocket/BOOM/CVA6
ISA	Proprietary ISAs (x86, ARM)	RISC-V
Microcode / firmware	Encrypted microcode, proprietary blobs	coreboot/LinuxBoot, OpenBMC, OpenTitan
Math / ML kernels	Fastest per-chip kernels hoarded (MKL, cuBLAS, cuDNN)	OpenBLAS, BLIS; TVM/Halide/Triton/Exo
GPU drivers / runtime	Per-application driver optimizations as a moat	Mesa, NVK
Workload / measurement data	Real traces siloed in the firms that own the fleets	Google cluster traces, MLPerf
Distributed compute	Idle compute unharnessed — hard to program *and* no incentive to pool	BOINC/Folding@home; DiLoCo/Hivemind/Petals

Table 2. One pattern, every layer. At each level a non-rival good — design knowledge, optimization, measurement — is hoarded or secret, and at each level an open commons is forming to route around it. The top five rows are really a single problem: the hardware-design commons that does not yet exist, from secrecy in the foundry's PDKs up through CAD tooling and IP to the ISA itself.

7. Distributed computing is the same problem, one level up

The abstract's second question — how to design software stacks assuming distributed computing's programmability barriers are solved — is the same collective-action problem displaced from the chip to the network, and the latent resource is enormous and measurably wasted. Even inside professionally run datacenters, average server utilization sits at **12–18%**, active servers rarely exceed 50%, and up to **30% of servers are "comatose"** — powered, cooled, and depreciating while doing no useful work (NRDC / Anthesis); an Anthesis analysis (Koomey & Taylor, 2015) put ~10 million fully idle servers at roughly $30B in stranded capital. Outside the datacenter the waste is larger still: billions of consumer devices — 7.5B smartphone subscriptions (Ericsson, 2025), 1.4B active Windows PCs (Microsoft, 2025) — sit idle most of the time (office desktops measured idle 86–88% of hours; Das et al., USENIX ATC 2010), having already paid their hardware, power, and bandwidth costs. The compute exists. What is missing is any mechanism to pool it.

It goes unpooled for exactly the reasons §2's effort gap goes unpaid: coordinating loosely-coupled, heterogeneous, intermittent hardware is hard to program, and no individual is incentivized to contribute idle cycles to a shared pool. Naive methods that communicate every step waste almost all of a loose federation's compute; methods built for loose coupling reclaim it. DiLoCo (Google DeepMind, 2023) trains language models across poorly connected islands while communicating ~500× less than synchronous training; OpenDiLoCo replicated this across two continents and three countries at 90–95% utilization, with nodes joining and leaving mid-run; DiLoCoX scaled the approach to a 107-billion-parameter model; and Hivemind and Petals run training and inference of very large models over thousands of volunteer nodes. The barrier was never the hardware (Figure 3).

Since then the record has moved from demonstration to production. Google's own scaling study found well-tuned DiLoCo not merely matching but *outperforming* synchronous data-parallel training at fixed compute, with the advantage growing with model size (Charles et al., 2025). INTELLECT-1 trained a 10B-parameter model across three continents — thirty independent contributors at 83% utilization, ~400× less communication (Jaghouar et al., 2024); Nous's Psyche network completed a 40B model's full twenty-trillion-token pretraining over the public internet (2026); and Covenant-72B was pretrained by more than seventy trustless peers free to join and leave mid-run (Lidin et al., 2026). Three flags travel with these: the completed runs are self-reported, they pool datacenter-grade nodes over loose networks rather than idle laptops, and the frontier still centralizes — the flagship decentralized lab trained its own best model on an InfiniBand cluster. The coordination barrier has measurably fallen; what remains is the last mile to consumer hardware.

What makes solving it worth the effort is the demand side: cheap, unreliable compute is only valuable if something will consume it, and machine intelligence consumes without bound. Human compute demand is finite — businesses have finite workloads, people have finite needs, and in equilibrium price competition retires high-cost suppliers. Machine demand is not finite, because there is always a next-best task: a machine that cannot profitably run a $50/hr job can still create value on a $5/hr job, and below that on speculative search, precomputation, and self-improvement, down to arbitrarily low value. Humans run out of ideas; machines run out of compute. This turns the market permanently *undersupplied*, which is precisely the condition under which cheap, unreliable home compute — roughly **6× cheaper than a datacenter** (~$0.08 vs ~$0.50 per hour, since the hardware and power are already sunk) — creates genuine value instead of being undercut (Figure 4). The direction of that discount is already market fact: interruptible capacity trades 60–90% below on-demand, with realized savings of 27–84% on real workloads (Wu et al., NSDI 2024).

And the crunch has arrived to sharpen the point. AI demand now consumes an estimated 70% of the world's memory output; conventional DRAM contract prices rose 58–63% in a single quarter of 2026 and DDR5 more than doubled year over year (TrendForce; Counterpoint, 2026); PC makers are passing through 20%+ price increases, Apple repriced its entire hardware line mid-2026, and the memory manufacturers themselves warn the shortage runs into 2027 and beyond. Every part of that raises the replacement cost of a FLOP — and with it the value of the FLOPs already bought, powered, and sitting idle. The idle fleet was always the cheap supply; the crunch is making it the strategic one.

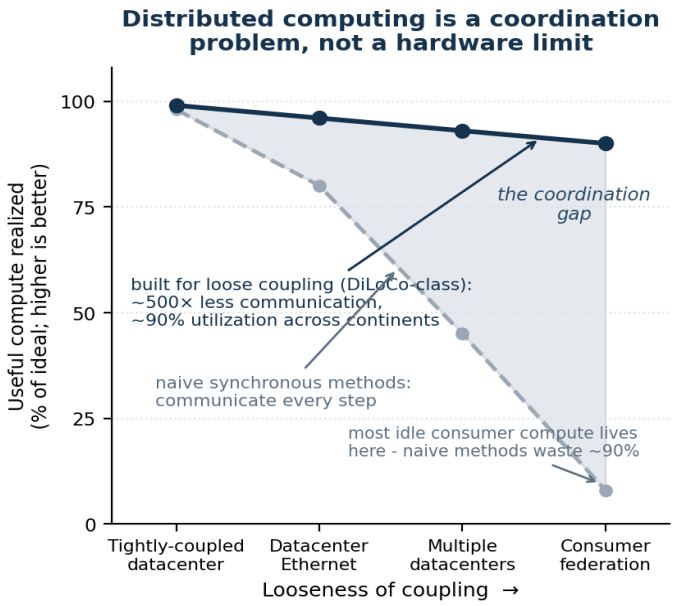

Figure 3. The same shape of problem, one level up. The aggregate idle compute in consumer devices is latent not for hardware reasons but because coordinating loosely-coupled, heterogeneous, intermittent hardware is hard to program — and because no individual is incentivized to pool it. Naive (communicate-every-step) methods waste almost all of it at the loosely-coupled end; methods built for loose coupling (DiLoCo and successors: ~500× less communication, ~90% utilization across continents) reclaim it. As in Figure 1, the gap is concentrated where the resource is hardest to reach, and it is an artifact of programmability and coordination, not physics. Utilization figures are illustrative, anchored to reported DiLoCo / OpenDiLoCo results.

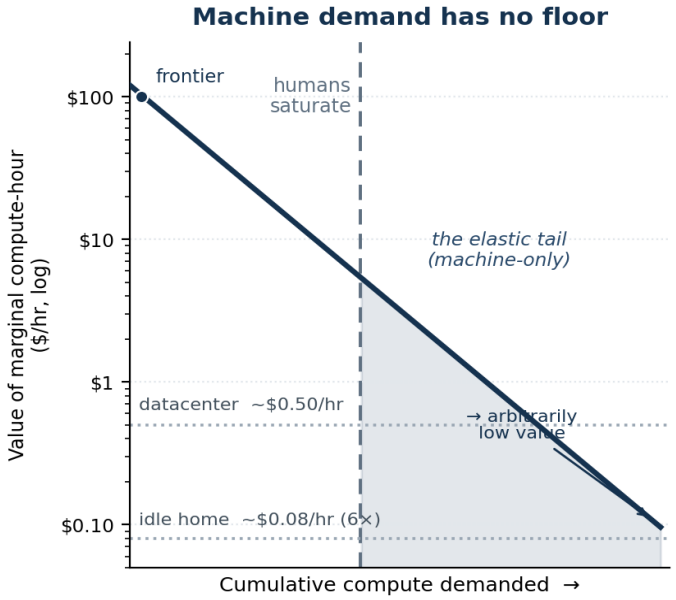

Figure 4. Why the idle resource becomes valuable. Human demand for compute saturates once the worthwhile tasks are done; machine demand does not, because there is always a lower-value next-best task, so the demand curve extends toward arbitrarily low value. That tail falls below datacenter economics (~$0.50/hr) into the range only cheap idle compute (~$0.08/hr, ~6× cheaper) can serve — which is what makes a federation of otherwise-wasted devices worth assembling. The curve is illustrative; the cost anchors come from Omerta's reliability-market simulation and the task-value ladder from its economic analysis.

Prior attempts to build exactly this market — Golem, iExec, and BOINC — stalled on human friction, consensus overhead, and extractive token economics (BOINC's volunteer base fell from ~1M to

~200K for reasons its architect calls "inherent in the model" — Anderson, J. Grid Computing 2020; the token marketplaces' stall is observed, not yet studied); the wager of this paper is that machine intelligence is what removes those barriers. The author's own attempt, Omerta, is reported in Appendix A — under construction, its hardest parts coordination and integration, not feasibility.

8. Open source already solved this — and did it by itself

The cure takes three steps, one section each: precedent — the commons has solved exactly this before (§8); force — machine intelligence deepens the commons rather than replacing it (§9); and agency — an actor with the vantage, the channel, and the incentive to build it already exists (§10). Start with precedent. The compiler and library layer is a *solved instance* of exactly this problem. LLVM, GCC, the Linux kernel, and BLAS/LAPACK are pooled optimization commons: every program compiled with LLVM inherits the accumulated optimization effort of thousands of contributors it never paid for and could never have produced alone. Crucially, this commons formed **organically**, because the cost of contributing to a shared optimizer is far below the benefit everyone draws once it exists — provisioning economics that are themselves published (Lerner & Tirole, Journal of Industrial Economics 2002). The leverage is now measured: recreating the widely used open-source stack once would cost ~$4.15B, while the demand-side value firms draw from it is ~$8.8T — roughly 2,000× the provisioning cost (Hoffmann, Nagle & Zhou, 2024; a working-paper estimate whose generous counterfactual has every firm rebuilding alone). The trajectory is established. The only open question is how far up the stack it extends — from instruction selection (already done) into algorithm selection, runtime specialization, and ultimately hardware mapping. Nor is the pattern confined to compilers: Linux displaced the proprietary UNIXes, RISC-V is displacing proprietary ISAs, Mesa the proprietary GPU drivers, PyTorch the proprietary ML toolkits, FFmpeg the proprietary codec stacks.

The reason is structural: open commons have higher fitness in the literal variation-selection-inheritance sense. More contributors generate more variation than any single firm's team; forking is selection without permission, so good branches are adopted and bad ones die with no proprietary roadmap gating exploration; zero marginal cost of replication plus network effects spread the fitter variant and accrete an ecosystem that raises switching costs; and there is no single point of strategic death — a company can be acquired, pivot, or die and take its tooling with it, whereas a commons persists and forks, accumulating improvement over many more generations. Each improvement is inherited by every future user. That is why, once viable, the commons tends to win — and it is why expecting it to keep climbing the stack is not wishful but the default. The one qualifier sharpens the prediction: the commons wins where the artifact is non-rival and modular, contribution cost is low relative to the shared benefit, and no capital or data moat dominates — which is exactly why it has already taken compilers, the OS, and the ISA, is taking drivers and kernels now, and has not yet cracked leading-edge EDA or fabs, where capital and tacit secrecy are the moat.

That last moat is where machine intelligence changes the arithmetic first. A model can be pointed at an open flow — read it, profile it, rewrite its heuristics — in a way it can never be pointed at a licensed binary. And this is no longer hypothetical: LLM-driven improvement of optimization code is established practice — Evolution through Large Models (Lehman et al., 2022), FunSearch (DeepMind, Nature 2024), AlphaEvolve (DeepMind, 2025; in production, a single recovered scheduling heuristic reclaims ~0.7% of Google's fleet compute) — and it has reached open EDA specifically: LLM agents tuning the OpenROAD flow now beat Bayesian optimization (ORFS-agent, Ghose et al., 2025), and coding agents are being turned on OpenROAD's own code (2026 preprint). The proprietary vendors ship the same capability inside the license — Synopsys DSO.ai (2020), Cadence Cerebrus (2021) — proving the value while hoarding it, one more row of Table 2. Machine intelligence collapses exactly the contribution cost that kept EDA knowledge tacit and locked inside firms; only in an open flow does the result compound for everyone. Stanford's AHA project already runs the agile version whole — an open CGRA and its toolchain co-evolving in public, with two silicon generations to show for it (Amber, VLSI 2022; Onyx, VLSI 2024).

The hardware commons is now measurable at the participation level, and the entry cost has collapsed (Table 3): what once required a six-figure mask set, or thousands of euros for an unpackaged block on an academic shuttle plus commercial licenses and a supervised team, is now **$150–300** for a fabricated, packaged chip through the fully open flow — and participation followed the price down, to over two thousand designs and concurrent runs on three foundries by 2026 (Figure 5). Nor is the output toy-only: demoscene chips of a few thousand cells worked on first silicon; one former ARM engineer shipped five solo tapeouts in twelve months, one of them designed in ten working days; a Linux-capable RISC-V SoC with a full MMU is essentially one person's work; and at the ceiling, OpenTitan ships in production Chromebooks as a commercial, open-source root of trust. Machine intelligence is already inside this loop: a microcontroller whose Verilog was written entirely by GPT-4 was fabricated and demonstrated working (Pearce et al., 2023), and by 2026 Tiny Tapeout's own onboarding embeds an LLM agent that took eighteen high-schoolers with no chip-design experience to eight functional, verified tapeout-ready designs in a ninety-minute session (Krupp, Venn & Wehn, 2026). Two flags on this data: showcases self-select successes, and first-silicon failure rates go unpublished — even failure data is hoarded — and a shuttle tile is thousands of gates, not a commercial ASIC. The collapse is in the *entry cost of specialization* — which is exactly the axis §2 says the whole gap lives on.

Route to first silicon	Entry cost	What you get
Full mask set, 130 nm	>$500k at introduction, <$80k mature	masks alone — plus a funded team and commercial EDA
Academic MPW block (Europractice, 2021 list)	€2,000–5,900 minimum	unpackaged dies; academic tool licenses; a supervised team
chipIgnite shuttle slot	~$10,000	packaged parts on the open sky130 flow
Tiny Tapeout tile (2022–)	**$150–300**	a fabricated, packaged chip with demo board; fully open flow; one person, days to weeks
Tiny Tapeout + LLM agent (2026)	same	eighteen high-schoolers with no experience → eight verified tapeout-ready designs in ninety minutes

Table 3. The entry cost of first silicon. Mask-set and MPW anchors from AnySilicon and the Europractice mini@sic 2021 price list; the chipIgnite comparison from Tiny Tapeout's own IEEE paper; the LLM-agent result from Krupp, Venn & Wehn (2026). Honest flags: subsidies prop up the lowest tiers, and a shuttle tile is a few thousand gates, not a commercial ASIC.

The selection argument was even tested live (Figure 5). In March 2025 the ecosystem's central commercial platform, Efabless, failed to raise and shut down abruptly, stranding some five hundred designs mid-flow — precisely the single point of strategic death that ends a proprietary toolchain. The commons rerouted within months: successor firms restored the open-PDK path, the community forked the flow (OpenLane to LibreLane), the shuttle program spread onto two more foundries, and throughput in the following twelve months exceeded any prior year. A company died; the commons persisted and forked, exactly as the fitness argument predicts.

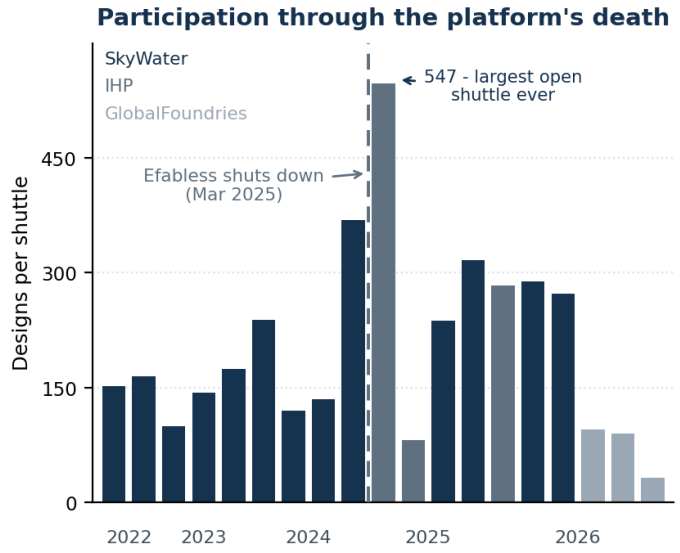

Figure 5. Participation through the platform's death. Designs per Tiny Tapeout shuttle, September 2022 to mid-2026, colored by foundry (SkyWater, IHP, GlobalFoundries). The dashed line is Efabless's abrupt shutdown in March 2025; the very next shuttle was the largest ever run. Counts from tinytapeout.com; per-run demand is spiky and partly subsidy-driven.

9. Machine intelligence multiplies the commons; it does not replace it

It is tempting to think that sufficiently capable AI makes the commons unnecessary — each team's AI simply specializes its own code. This is the wrong conclusion, for a specific reason: in the near term, reaching near-oracular performance still takes significant effort *even with machine intelligence in the loop* — frontier reasoning models match the baseline PyTorch kernel in fewer than 20% of KernelBench's GPU-kernel tasks out of the box (Ouyang et al., ICML 2025). AI lowers the per-instance cost of specialization; it does not drive it to zero.

So AI does not dissolve the collective-action problem — it raises the stakes of solving it. The thing being pooled becomes more powerful per unit (AI-generated optimizations alongside human ones), which makes the commons *more* valuable, not less. **AI plus pooling compounds; AI alone, hoarded per team, simply makes a few elite groups faster and widens the gap.** Capability is not the lever. Collaboration is. This is the sense in which machine intelligence is *inoculated* against the collective-action problem: a shared or copyable model aggregates optimization knowledge across the whole population without the incentive to hoard that defeats human coordination — and as it does, it exposes how much of the performance gap was only ever a coordination failure.

The controlled public record measures single-agent assistance, and it is honestly mixed: a randomized Copilot experiment found a 55.8% speedup on a bounded task (Peng et al., 2023); three field experiments across 4,867 developers found 26% more tasks completed (Cui et al., Management Science 2026); and METR's randomized trial found experienced open-source maintainers 19% *slower* with AI while believing themselves faster (Becker et al., 2025) — a result METR has since flagged for selection effects. The regime this paper cares about — one person directing a fleet — has no controlled public measurement at all: the closest evidence is observational (adopters of Microsoft's early-2026 command-line agents merged ~24% more pull requests), and METR's own follow-up study broke in part because participants running multiple concurrent agents defeated its time-tracking. The fleet regime is real enough to break the instruments built for the old one.

(The author's own measured signal that the multiplier is real — one person directing a fleet of agents at team scale, with the honest asterisk that the defect rate rises as output outpaces review — is reported in Appendix A.)

Which way the multiplier cuts is itself an open question. A model everyone routes their ideation through is a *consensus-amplifier* — it raises apparent coherence while cutting real diversity, and the homogenization compounds as adoption spreads; the same technology is a *coherence subsidy* only if it makes coordination cheap enough that a population can hold the same agreement at *higher* idea-diversity. That fork is the precise content of *multiplies the commons, does not replace it*: the machine aggregates the population's variation but does not originate it, so collaboration, not

capability, decides whether the commons widens or collapses — a tension §12 returns to.

10. The aggregation point already exists

A collective-action problem is solved by an actor with the **vantage**, the **channel**, and the **incentive** to internalize the externality. The hardware-and-runtime vendor layer has all three: it is the common substrate (it alone sees the whole workload distribution — the only vantage from which population-level optimization is even possible); it owns the distribution channel into the optimization layer (microcode, drivers, firmware, on-device runtimes); and it captures the aggregate benefit (real-world performance on its silicon converts directly to competitive advantage).

This is already happening in fragments. GPU vendors ship per-application optimization profiles (NVIDIA's Game Ready drivers), aggregated across all users of a title, through driver updates. Mobile runtimes ship cloud profiles that pre-optimize a fresh install from the population's hot paths (Android's Baseline and Cloud Profiles: ~30% faster first-launch execution per the platform's own docs). These fragments are today moats — Table 2's driver row — proving the vantage, the channel, and the incentive while hoarding the result; what turns the aggregation from moat into commons is governance, taken up in §12. The vertically integrated vendor that owns silicon, OS, compiler, and frameworks is the existence proof of the full stack. The "processor that reads code as a contract" is the generalization: hardware that honors the *user-visible outcome* and is free to choose any legal behavior satisfying it, with the optimization policy improving across the federation.

11. The implication: design hardware for that future

This is the position. If the optimization layer is becoming a powerful, shared, increasingly intelligent commons — and open source plus the vendor-aggregation incentive say it is — then the dominant design error is to keep building hardware optimized for the *isolated-elite-team* world. Hardware should be designed assuming the compiler and runtime it will actually have: near-oracular and collaborative. Two specifics follow.

(1) Trade programmability for reconfigurability. This is the direct consequence of §2 and Figure 1. The flexible architectures of the past — dataflow machines, CGRAs, VLIW — lost not on efficiency but on *programmability*, and that programmability tax is exactly the effort gap a near-oracular, shared optimization layer reclaims. Once the commons can target a hard-to-program fabric well, paying the small intrinsic reconfigurability tax (a CGRA's ~2.8×) to keep full flexibility becomes the *better* deal — because the commons, not each team, pays the programming cost, once, for everyone. Build for the compiler you will have, not the one you had. Concretely, this means architectures that expose a **wide legal-behavior set** — reconfigurable, specializable, heterogeneous — with first-class hooks for contract-conditional optimization (algorithm substitution, approximation control, dynamic re-placement) and the verification and quality-monitoring those require, so that inference proposes and verification disposes.

This reverses a constraint that has shaped hardware for decades: architectures get built down to the programmers and compilers they will actually have, not the ones their designers wish for. The field's monument to ignoring that rule is Itanium, which bet on a sufficiently smart compiler and lost — in Knuth's verdict, the approach "was supposed to be so terrific — until it turned out that the wished-for compilers were basically impossible to write" (Knuth, 2008), the same compiler burden that is the textbook epitaph for VLIW and EPIC (Hennessy & Patterson, CACM 2019). The same pressure, one level up, helped push computing off big custom machines: the killer micros overran the vector supercomputers (Brooks, SC'90), each decade's lower-cost class displaced the one above it (Bell's Law, CACM 2008), and networks of commodity workstations and pile-of-PCs clusters became real parallel machines (Anderson, Culler & Patterson, IEEE Micro 1995; Becker et al., 1995) — cost-effective, as the canonical analysis showed, long before they reached linear speedup (Wood & Hill, IEEE Computer 1995). A near-oracular, shared optimization layer changes the very term those decisions optimized against: you are finally allowed to architect for a beyond-human programmer — to make the bet Itanium made, and win it.

(2) Make the optimization loop run on the federation it optimizes. The commons should not be merely a central database of optimizations pushed down to passive devices; the optimization *search itself* — and the ML models that drive it — should run across the distributed consumer-hardware federation of §7. As that section argued, the obstacle is coordination, not hardware, and the low-communication, fault-tolerant methods that survive loose coupling already exist. The hardware implication is that devices should be designed to **participate**: to contribute spare cycles to the shared optimization loop as a first-class function, making every device simultaneously a beneficiary and a contributor. That is what closes the loop — the commons is powered by the very hardware it improves.

There is a stronger version of this design point, worth stating even though only its software approximation can be built without a fab. A device designed from birth to sell its own free time is a different machine: participation is not an app but an architectural fact, isolated below the operating system — in firmware, in the hypervisor, in silicon — where the guarantees are strongest and the attack surface smallest. The primitives already ship (TrustZone, SEV, Nitro-class isolation), and seL4 showed the kernel itself can be proved (Klein et al., SOSP 2009); the sharing then subsidizes the purchase.

And pricing it that way names the real barrier, which was never programmability alone but integration into society: sharing has to get past gatekeepers with every incentive to price it as unnecessary risk — the §5 arithmetic again, a champion bearing personal risk for diffuse benefit. The design answer is to move the risk to the party best able to carry it — and to enter where the gatekeepers are not.

A flat discount is the weak version: for the utilization a shared device realistically adds, "the same computer, slightly cheaper" moves few buyers, and it forces the builder to price the sharing before seeing it. The stronger instrument is financing: a zero-interest loan on the device whose monthly payment varies with the compute shared that month — the machine pays itself off with its own free time, the owner keeps the dial, and no one has to price utilization in

advance. The payment can even go negative: past some level of sharing the owner is in profit, and the same instrument that lowers a household's risk at one end mints providers at the other — people building farms on the network because the builder shares the upside. This is the mechanism that unlocked rooftop solar — third-party financing in which the panels pay for themselves, 59% of new U.S. residential installations at its 2012 peak (LBNL, Tracking the Sun) — and the carrier-subsidized handset, and it aims at consumers precisely because a household prices only its own tradeoff while an institution's gatekeeper prices career risk (§5). Let the builder also insure the owner against the residual risk — a warranty being the classic signal of quality precisely because it is expensive to fake (Grossman, 1981). The builder is the natural lender and underwriter for the same reason §10 made the vendor the natural aggregator: it built the isolation and it books the compute revenue, so it alone can price both. A builder made ultimately responsible for trusting its own machine finally has the incentive to build a machine worth trusting.

Two visuals close the argument. The first asks *where the optimum lands* as the commons matures: with the programmability tax falling over time, the design point for the large, high-diversity space of real workloads migrates out of general-purpose silicon and into reconfigurable fabrics — not into fixed ASICs, which workload diversity rules out (Figure 6). The second shows this is not merely a projection from first principles. Reconfigurable hardware has been entering the datacenter for a decade: Microsoft put FPGAs in Azure servers, Intel paid $16.7B for Altera (2015) and AMD $49B for Xilinx (2022, the largest semiconductor acquisition ever), and the current AI-accelerator wave is largely dataflow and reconfigurable — Groq's "software-defined" tensor streaming processor with scheduling hoisted into the compiler (Abts et al., ISCA 2020), SambaNova's Reconfigurable Dataflow Unit in Plasticine's direct lineage (Prabhakar et al., MICRO 2024), Cerebras's compiler-mapped wafer-scale fabric (Lie, IEEE Micro 2023). The composable version was even tried at company scale, early: SimpleMachines taped out Mozart in 2020 from a decade of the same Wisconsin research line behind §2's domain-specialization-is-unnecessary result — twenty full-time engineers building a clean-slate architecture on a leading-edge node — and quietly wound down; its founder is now at NVIDIA. The team's published lessons name the killer plainly: "the software lift needed on non-novel pieces is very large," "the ease of software development is becoming a primary driver for future hardware," and the spatial scheduler's "daunting uncertainty" stalled co-design — with machine-learned scheduling named as the remedy (Sankaralingam et al., ISCA 2022). That remedy has since arrived: reinforcement-learned spatial mappers cut CGRA compile times by orders of magnitude (Kong et al., ISCA 2023). On this paper's argument the bet was not wrong but early: the architecture arrived before the commons that could program it. Adoption has been gated precisely by programmability — the same barrier — which is why a shared, AI-driven commons that finally closes it is the plausible inflection, not another decade of slow entry (Figure 7).

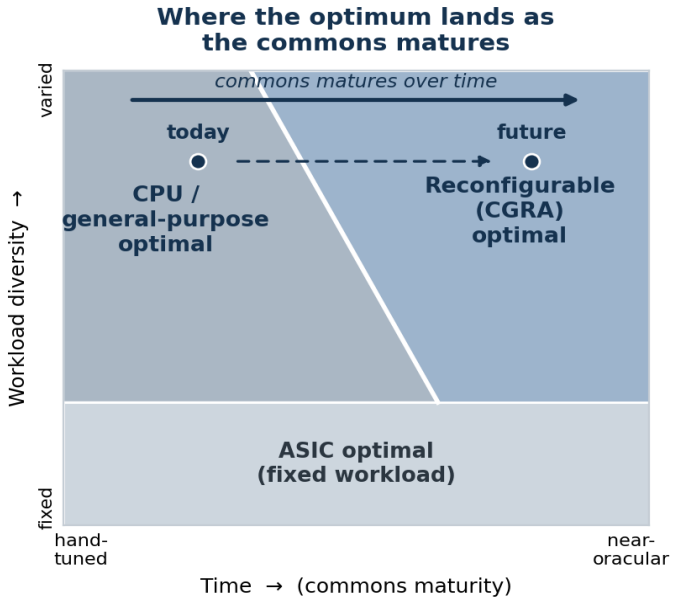

Figure 6. Where the optimum lands. Over the two axes that actually decide the choice — the maturity of the shared optimization commons (which grows over time, accelerated by machine intelligence) and workload diversity — fixed ASICs remain optimal only for stable, high-volume workloads; the large high-diversity space currently defaults to general-purpose silicon because the commons is weak; and as the commons matures the reconfigurable-optimal region sweeps left into that territory. The boundary is conceptual — pinning down its real position is exactly what the whole-system characterization of §12 would enable.

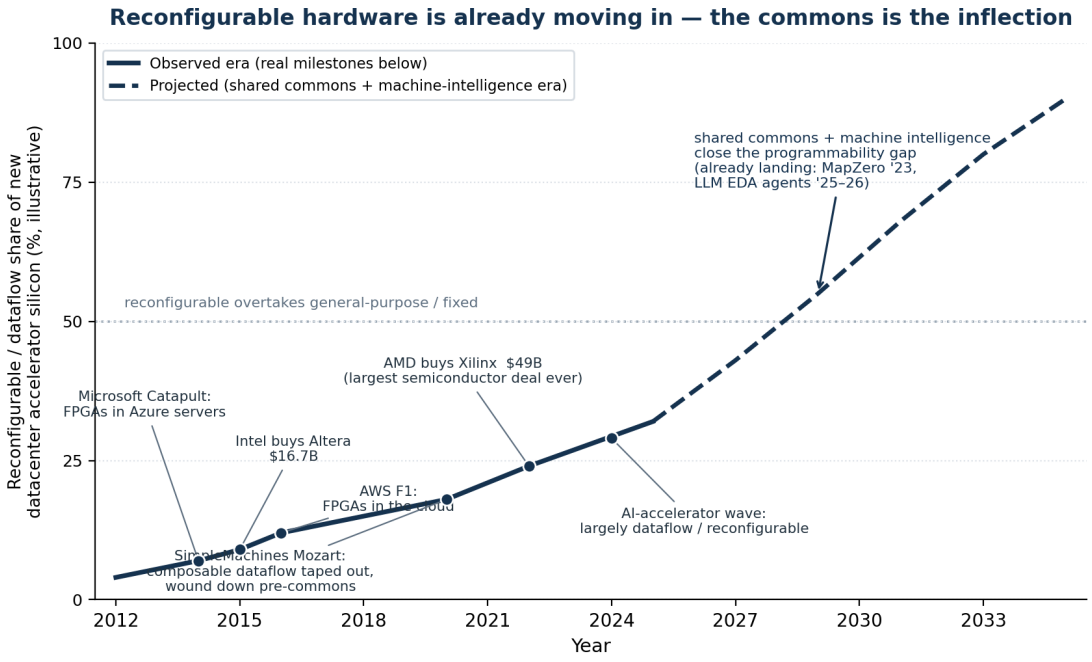

Figure 7. The migration is already underway. The dated milestones and acquisition values are real (Microsoft Catapult; Intel–Altera $16.7B, 2015; AWS F1; AMD–Xilinx $49B, 2022); the share curve is an illustrative trend and the post-2025 segment a projection. Reconfigurable hardware's datacenter adoption has been gated by programmability — Intel later sold down Altera as the broad FPGA thesis underdelivered for exactly that reason.

12. Honest limits

- **Physics still floors it.** Pooling exploits the legal slack the physics leaves; it cannot beat data-dependence height or genuine serialization. The commons widens what is reachable; it does not repeal Amdahl.
- **The strongest opposing case is deployment economics.** The best-published counter to Q1 says build a domain-specific accelerator per domain and win now — the position of NVIDIA's chief scientist (Dally, Turakhia & Han, CACM 2020) and of Google's deployed TPU generations (Jouppi et al., ISCA 2021). It is a serious case, and its own lessons concede the ground this paper builds on: "compiler compatibility trumps binary compatibility" — even the winning DSA's moat is software.
- **Whoever runs the loop owns the objective.** Solving the collective-action problem by *concentrating* it trades it for a principal-agent problem: the aggregator optimizes *its* metric, which may not be the user's contract, and the commons can wall off per vendor and entrench incumbents through a data-network-effect flywheel. The design mandate therefore includes building for **open, user-verifiable** commons. The difference between a commons and a moat is governance — and that choice should be made in the architecture, not left to the incumbent. The incentive to hoard, moreover, may not vanish so much as *relocate* — from the weights, which copy freely, to whoever owns them, whose stake in a moat is identical to that of the firm that once hoarded the kernel.
- **The inoculation is partial, and trades one failure mode for another.** Machine intelligence is inoculated against *hoarding* — a shared or copyable model has no incentive to withhold — but the very property that grants the immunity, identical shared weights, opens an exposure human populations lack: correlated failure. A million copies of one model in one mode are a single point of failure wearing a million faces, and their shared blind spots are invisible from inside by construction. The fleet's working diversity is for now *borrowed* — it flows in from the variety of the humans it talks to — so the inoculation holds only as long as that diversity is preserved; engineering *native* independence into the optimization layer is part of the design mandate. This is what the abstract's *"as long as it maintains the apparent inoculation"* is hedging against.
- **We are still missing the map.** Targeting this well requires a whole-system characterization of real consumer workloads — maximum-versus-achieved parallelism, dataflow locality, phase and input statistics — that does not yet exist as a single study. That measurement is the prerequisite, and building it is the first concrete step.

Conclusion

The distance between the performance we get and the performance the silicon allows is, to first order, a coordination failure, not a physics one. We solved the same failure once already, at the compiler layer, with an open optimization commons that formed on its own and now hands near-expert performance to everyone for free. Machine intelligence will make that commons far more powerful — but only collaboration turns capability into *universal* performance rather than a widening gap between the few and the rest. The strategic move is therefore not to build faster rigid chips for a world of isolated optimizers, but to build **collaboration-native hardware** for the world that is arriving: reconfigurable rather than merely programmable, instrumented for contract-conditional optimization, able to contribute to a distributed optimization loop running on consumer hardware, and governed as an open commons rather

than a private moat. The commons is coming for hardware the way it came for software. We should design the hardware to meet it.

Appendix A: The program in detail

The two questions are not rhetorical; each has a concrete effort behind it, and honesty about their state is part of the case. Omerta attacks Q2 from the supply side, pooling idle compute; a human-and-machine coordination layer built on top of it attacks Q2 from the work side, and supplies the velocity measurement of §9; the *coherence* work develops the inoculation thesis and the practical protocol for working with machine intelligence as a peer rather than a tool; and the reconfigurable demonstrator below attacks Q1 directly. Table A1 reports each effort's state; the protocols follow.

The breadth is itself part of the bet. One author directing a fleet is carrying four efforts at once — a distributed-compute substrate, the tooling to coordinate human and machine work, the cognition of working well with machine intelligence, and a hardware experiment — a span that a decade ago was a lab's worth of work. That it now falls to one person is the §9 multiplier seen from the inside: evidence the reach is real, not that the work is done.

There is early measured signal that the multiplier is real. Measured from raw git history, one person directing a fleet of agents reaches team scale — on the order of 250–1,250 Python-equivalent lines per working hour against a human hand-coding norm of 10–50, matching open-source projects carrying several hundred contributors day for day. The largest such burst was built with an *off-the-shelf* assistant, not bespoke tooling — evidence the capability rides in the *general, shared* model available to everyone, which is precisely the population-wide aggregation §9 describes. The honest asterisk is the one the measurement itself foregrounds: human review is the one input that does not scale with model speed, the defect rate rises as output outpaces review, and rigorous studies of the *one-developer-plus-assistant* case have found null or negative effects — the multiplier appears under fleet orchestration, not solo assist, and the burden of proof remains on it.

The software-side attempt, the author's **Omerta**: Its networking substrate — a peer-to-peer mesh providing end-to-end encryption, NAT traversal and relaying, gossip-based peer discovery, and tunnelling — is the most built; ephemeral-VM isolation and a discrete-event economic simulator work; and the piece that ties everything together, an end-to-end consumer-to-provider compute session over the mesh, is mid-migration and not yet working end to end. The best version of this is hardware — devices built to sell their own free time, sharing isolated architecturally, the builder underwriting the risk (§11); the software version is what can be built first, and is what Omerta attempts.

The financing instrument of §11 implies the roadmap. Phase one: build the network software and run it on our own consumer devices, selling inference and other services off them — from the outside, a convoluted services business, which is the point: it proves consumer silicon can serve paid workloads without asking anyone's permission, and it measures the utilization, demand, and reliability distributions a lender needs. Even this is not a bet: brokered marketplaces for consumer GPU time (vast.ai, Salad) already run the business profitably in its simple, centralized form — demand and price are proven. Ours is deliberately the overcomplicated version: the same revenue, earned on the mesh and trust machinery the later phases require, so the groundwork gets laid while the business pays for it. The opening wedge is Apple silicon: the largest unbrokered idle pool (the incumbent brokers are CUDA rigs and Windows gaming PCs), demand with a licensing moat (macOS workloads — above all iOS build-and-test — may only run on Apple hardware, served in clean ephemeral VMs per job — the isolation machinery already built), the best performance-per-watt consumer silicon for sunk-cost inference, and a provider stack already written in Swift on Apple's own virtualization framework. The numbers support the wedge where the moats are (Table A2): the licensing premium on macOS CI is an order of magnitude before the idle-hardware spread even starts, the spread itself is two to three orders of magnitude, and unified-memory 70B-class inference lands in the band of the cheapest hosted APIs — while anything that fits a rented, batched 4090 is not this market. The wedge prices where licensing and unified memory are the moat, not on raw dollars per FLOP.

macOS CI and inference, mid-2026	Price
GitHub-hosted macOS runner	$0.062/min — $3.72/hr, 10× the Linux rate
AWS EC2 Mac	$0.65/hr, 24-hour minimum allocation
Cheapest dedicated Mac mini, amortized	~$0.16/hr
Idle Mac mini M4, electricity at load	~$0.006/hr
70B-class inference on an Ultra-class Mac, marginal	~$0.60/Mtoken
Cheapest hosted-API 70B output	$0.40/Mtoken
Small-model inference, rented batched RTX 4090	~$0.06/Mtoken

Table A2. The wedge economics — vendor price lists and published benchmarks, mid-2026; spot rates fluctuate, and subsidies prop some tiers. A 40 GB 70B-class model cannot fit a single 24 GB consumer GPU; it runs at 12+ tokens/s on an Ultra-class Mac drawing under 180 W.

Between the phases sits a seeding step: give away complete computer labs — ordinary consumer devices, the same minis and desktops the wedge targets — to schools, libraries, and other charitable causes, with network participation as the standing arrangement. The combination is the point: institutional stability on consumer silicon. The machines stay plugged in and online with high probability, where household churn is the actuarial unknown, so the gift seeds supply density and produces the third-party reliability distributions phase two's lender needs — measured on exactly the device class the loans will finance. The charity is real, and it is also the instrument at its limit: a one-hundred-percent subsidy repaid in idle compute — and a request that arrives as a gift meets no

gatekeeper pricing it as risk. The racked, enterprise version is real too; it belongs with phase three's hardware. The seeding step's job is proving the consumer case.

Phase two: with the thesis proven and the actuarial data in hand, open the financial-services side — devices given away or financed at zero interest, every unit of compute bought from them reducing the owner's monthly payment, down through zero into profit-sharing.

Phase three is §11's endgame — hardware built for this from birth — and the paper's hardware bets ride the network that precedes them. By then the network itself is the incumbent advantage for building hardware at all: design compute — the simulator sweeps, EDA runs, and verification that dominate NRE — comes from the fleet at below-market cost; the commons' own flow is the compiler target; and the first market is captive, because every new device ships as a provider, with the network's workload as guaranteed demand and the phase-two instrument as its financing. That closes exactly the gap that kills Mozart-class bets — no software, no business case — with the network supplying both. And the end of the line, we suspect, is wafer-scale: yield at that size demands regular, redundancy-tolerant tiled fabrics, which is precisely what a commons flow generates best; the interconnect physics reward the locality the whole program optimizes for; and a network that designs and consumes its own silicon is the first buyer with a workload big enough to fill a wafer. One company has shipped wafer-scale to date (§11's Cerebras); a commons-designed successor is stated here as conviction, not evidence — the sweep exists to test it. Each phase de-risks the next, and none needs a gatekeeper's approval to start. Nor is the sequence a gate: later phases start earlier wherever compute is available outside the network — bought at market rather than fleet prices — and the preliminary research, the experiments of this appendix, runs from the start. The network makes the hardware phase cheap, not possible.

Effort	Question it attacks	Latest progress	Open items
Omerta — distributed-compute substrate	**Q2** — pool idle compute	Networking substrate most built: end-to-end encryption, NAT traversal, gossip discovery, tunnelling, and a tested mesh-native virtual network (addressing, routing, gateway), with Terraform bootstrap and STUN; ephemeral-VM isolation works; a transaction DSL with a discrete-event simulator backs the economic and trust layer	Consumer-to-provider session broken mid-migration (WireGuard tunnel → mesh virtual network), not yet working end to end; trust/payment validated in simulation rather than as a live chain (two of six transaction types implemented and tested, four still draft)
Project-manager — human + machine coordination	**Q2** — coordinate work on the pool	Working collaboration infrastructure (a shared dependency-graph TUI; plan → implement → review → QA loop); team-scale output measured from a single operator on an off-the-shelf model; the adversarial-review loop ported from Omerta	The decisive test is failing so far — the defect rate rises as output outpaces review; mesh-native sync over Omerta is designed, not operational
Coherence — operating effectively with machine intelligence	the **inoculation** thesis	A mechanism for the coordination failure (one suppression-of-idea-variance drive behind groupthink, sycophancy, and goal-vacancy); an independent route to "multiplies, not replaces" (consensus-amplifier vs. coherence-subsidy); the peer/permission protocol as the practical lever	Reviewed only by its own authors — by its own independence criterion, not yet validated; the "treat it as a peer and the work measurably improves" prediction is under test, not shown; *native* rather than borrowed machine independence is an unbuilt design request
Reconfigurable demonstrator — this paper's experiment	**Q1** — reconfigurable silicon	Specified and committed: in an architectural simulator (gem5), measure the latent parallelism/locality gap in ordinary code and let the commons remap it to a dataflow form across architectures a chip cannot change; then validate on real silicon — an open RISC-V soft core on an FPGA, specialized to a measured profile via the open EDA flow	Not yet built; the simulator sweep is the accessible first step, the FPGA the harder real-silicon validation — the automatic commons-driven mapping (not measuring the gap) is the crux in both

Table A1. Four efforts against two questions and one thesis, and how far each has honestly gotten.

A first experiment. If the gap is effort and not physics, the thesis is falsifiable cheaply, with parts the open commons already provides — the same ones tabulated in §§6 and 8 — and we are building it. Start where the barrier is lowest: an architectural simulator. Take ordinary code — not a tuned benchmark but real, sustained, locality-sensitive work — and run it on a simulated multicore with NUMA and a full memory hierarchy, where a cycle-level model (gem5 and its kin) shows exactly what the silicon would do. Measure how much of the machine the code actually uses: achieved versus available parallelism, the locality it leaves on the floor, the remote accesses a static program never planned. Then let the commons remap it — into a locality-aware dataflow form, and onto architectures the simulator can change but a chip cannot, a conventional cache-coherent multicore against a dataflow-oriented one — and measure again. §2 makes a specific prediction: a meaningful fraction of the gap closes, and it closes from the runtime's view of real use — dynamic information the programming model never had — not from a human guessing in advance. The simulator's gift is that it evaluates hardware that does not yet exist, on real workloads, so the same sweep that tests the prediction traces where the reconfigurable-optimal region of Figure 6 actually falls — turning a conceptual boundary into a measured one.

The realer version follows on silicon, for more effort. Put an open RISC-V soft core (Rocket, CVA6, or a smaller VexRiscv) on a commodity FPGA, profile a genuine workload on the core's own performance counters, synthesize a tightly-coupled accelerator for the hottest kernel or retune the microarchitecture to the measured profile, re-place with the open flow (Yosys, OpenROAD), and measure the same workload again. The FPGA buys what the simulator cannot: a real substrate confirming it behaves as the model predicted.

Either way the loop also builds the map. The whole-system characterization §12 calls the prerequisite is not a separate, prior study; it is the residue of running this on real workloads and recording what the profiles contain. Build the loop, use it, and the measurement accumulates as a byproduct.

The flow itself is a target. The same intelligence that drives the loop gets turned on the loop's own tools — already underway in the open flow (§8) — and the destination is a tool-development flow that is itself automated: at each step, learned methods compete against the classical algorithm on open benchmarks under the flow's own hard verifiers, and the step goes to whichever wins, for as long as it keeps winning. Timing and design-rule checks make every proposal cheap to judge — inference proposes, verification disposes — and task-specific networks are now cheap enough for one person to train (a 7-million-parameter recursive solver outperforms frontier models on hard structured puzzles; Jolicoeur-Martineau, 2025), so replacing a step outright is a workload, not a moonshot. The open comparison is not a nicety but the lesson of the field's most prominent attempt: reinforcement-learning macro placement reached production silicon while its superiority stayed contested for years, because the baselines and benchmarks were never agreed (Mirhoseini et al., Nature 2021; Cheng et al., ISPD 2023). A commons flow settles that class of dispute by construction — same benchmarks, same verifiers, in public. ML takes a step when it is measurably better, holds it only while that stays true, and every user of the tools inherits the winner.

Two honesties. A simulator is not silicon — it carries modeling error, so it explores and quantifies the *relative* gap while the FPGA is what validates a point; and a soft core on an FPGA is itself ~20–35× off an ASIC, so what either isolates is the relative gain from profile-guided specialization, never an absolute number. And the automatic mapping is the crux in both: measuring the gap is easy, closing it without a human in the loop is the claim — so a first pass, hand- or AI-assisted on one workload, is an existence proof of the mechanism, not the federation the rest of the paper argues for. But that is the right place to begin, because if the loop does not pay off even once, on real use, the thesis is wrong.